

Because multiple variables can refer to the same object, that object can be “stored” in multiple variables. Multiple boxes can’t store the same object, so it might be easier for you to use the label metaphor instead. Ned Batchelder’s PyCon 2015 talk, “Facts and Myths about Python Names and Values” has more information on this topic at <https://youtu.be/AEJHKGk9ns>.

Without understanding that the `=` assignment operator always copies the reference, not the object, you might introduce bugs by thinking that you’re making a duplicate copy of an object when really you’re copying the reference to the original object. Fortunately, this isn’t an issue for immutable values like integers, strings, and tuples for reasons that I’ll explain in “Mutable and Immutable” on page 114.

You can use the `is` operator to compare whether two objects have the same identity. In contrast, the `==` operator checks only whether object values are the same. You can consider `x is y` to be shorthand for `id(x) == id(y)`. Enter the following into the interactive shell to see the difference:

```
>>> spam = {'name': 'Zophie'}
❶ >>> eggs = spam
>>> spam is eggs
True
>>> spam == eggs
True
❷ >>> bacon = {'name': 'Zophie'}
>>> spam == bacon
True
>>> spam is bacon
False
```

The variables `spam` and `eggs` refer to the same dictionary object ❶, so their identities and values are the same. But `bacon` refers to a separate dictionary object ❷, even though it contains data identical to `spam` and `eggs`. The identical data means `bacon` has the same value as `spam` and `eggs`, but they’re two different objects with two different identities.

Items

In Python, an object that is inside a container object, like a list or dictionary, is also called an *item* or an *element*. For example, the strings in the list `['dog', 'cat', 'moose']` are objects but are also called items.